

ARGUS

Concurrent Event Investigation & Anomaly Detection Engine

PROJECT REPORT

Build Your Own Project — Java

"Local Security Event Analysis, Explainable Anomaly Detection and User Investigation"

Prepared By:

School of Computing Science and Engineering
VIT Bhopal University

Technology: Java 17+ | Java Standard Library | CLI

Version: v1.0 — September 2026

1. Introduction

In modern digital environments, the volume of system logs and security events generated daily is staggering. These logs record crucial system interactions, configurations, access attempts, and administrative actions. However, manually inspecting these logs to identify suspicious activity or track down the actions of a specific user is extremely inefficient and prone to human error. Without an automated mechanism, suspicious patterns—such as repeated failed authentication attempts or critical policy changes—can easily go unnoticed until it is too late.

To address this challenge, the **ARGUS** (Concurrent Event Investigation & Anomaly Detection Engine) project was developed. ARGUS is a lightweight, local Java command-line application designed for security-event analysis, transparent anomaly detection, and focused user investigation. The system provides a centralized tool to ingest validated security events, persist them locally, and analyze them concurrently against deterministic rules to highlight potential security risks.

The core philosophy of ARGUS is simplicity, transparency, and determinism. While enterprise environments often rely on massive, distributed Security Information and Event Management (SIEM) systems backed by machine learning models, smaller teams or individual evaluators require straightforward tools that do not depend on external infrastructure, cloud integrations, or complex deployments. Therefore, Java was selected as the optimal implementation language. Its robust standard library, strong object-oriented capabilities, predictable file I/O operations, and native concurrency primitives (such as the `ExecutorService` framework) make it ideal for building a reliable, self-contained application.

Target Users:

- **Analyst / Administrator:** Users who need a local, lightweight tool to parse logs, assess security anomalies, and investigate the timeline of actions performed by specific users.
- **Evaluator:** Academic or technical reviewers assessing the application of object-oriented design and concurrent programming in Java.

Scope Note: ARGUS is deliberately constructed as an academic Minimum Viable Product (MVP). It operates on a local file-based storage model and must not be conflated with a production-grade SIEM, enterprise security platform, real-time monitoring platform, or machine learning system.

2. Problem Statement

Problem Definition: Security incidents often originate from subtle, repeated anomalies or specific high-severity actions scattered within large volumes of event data. For individual administrators, small operational teams, or developers operating without enterprise-grade SIEM infrastructure, analyzing raw, untyped log files manually is prohibitively difficult. Crucial indicators of compromise, such as persistent failed logins or unauthorized configuration changes, are frequently overlooked due to log fatigue and a lack of localized analysis tools.

Proposed Solution: ARGUS resolves this by providing a self-contained, offline execution engine that ingests structured log data, validates its integrity, and processes it concurrently against predefined, explainable detection rules. By executing locally via the command-line interface (CLI) and storing data in a structured flat file, ARGUS entirely bypasses the need for complex database setups or external dependencies.

Motivation: Investigators require findings that are easily understandable. A black-box ML model is often inappropriate for simple audit trails because it cannot explicitly explain *why* a risk score was assigned. ARGUS is motivated by the need for transparent, rules-based risk assessment where every flagged anomaly is backed by a verifiable event footprint.

Scope: The scope of ARGUS v1.0 is strictly limited to localized file storage (CSV format), static rule definitions, command-line execution, and bounded concurrency using standard Java multithreading techniques. It focuses explicitly on deterministic analysis rather than predictive forecasting.

3. Objectives

The development of the ARGUS project was guided by the following core objectives:

1. **Persist Valid Security Events:** Ensure logs are durably saved to a local file system for subsequent analysis.
2. **Validate Event Data:** Guarantee data integrity by preventing blank fields, rejecting unsupported event types, and escaping unsafe characters prior to storage.
3. **Detect Suspicious Activity:** Apply deterministic, explainable rules to identify potential threats.
4. **Execute Concurrently:** Leverage Java's multithreading capabilities to run independent detection rules concurrently, demonstrating modern threading patterns.
5. **Generate Explainable Findings:** Produce clear, human-readable outputs detailing the exact criteria that triggered an anomaly.
6. **Provide Investigation Timeline:** Allow administrators to filter and chronologically trace a single user's actions.
7. **Calculate Risk Score:** Aggregate individual anomalies into a summarized numerical risk score per user.
8. **Demonstrate Java OOP:** Apply robust object-oriented principles, including encapsulation, abstraction, interfaces, polymorphism, and immutable data structures.
9. **Terminal-Based Execution:** Ensure the application is entirely operable via a command-line interface (interactive and single-execution modes).
10. **Provide Repeatable Testing:** Ensure the core logic is verifiable through a dependency-free, standard Java executable test suite.

4. Functional Requirements

The system must satisfy the following functional requirements to fulfill its objectives:

ID	Requirement	Input	Processing	Output / Implementing Class
FR-01	Record Events	Event attributes (user, type, severity, source, message)	Create an immutable <code>Event</code> object with a generated timestamp and unique ID.	<code>Event</code> model instance.
FR-02	Validate Input	Raw Event object	Check for non-blank required fields, supported vocabulary, and restricted delimiters.	Boolean validation result via <code>EventValidator</code> .
FR-03	Store/Retrieve Events	Validated Event list	Format data into pipe-delimited records and interact with the local filesystem.	<code>data/events.csv</code> update via <code>EventStore</code> .
FR-04	Detect Anomalies	List of parsed Events	Evaluate events against logical conditions (e.g., 3 failed logins in 15 mins).	List of <code>DetectionResult</code> via <code>EventProcessor</code> .
FR-05	Investigate User	User identifier	Filter the global event list, sort chronologically, and aggregate risk points.	Timeline output via <code>InvestigationService</code> .
FR-06	Command-Line Execution	Terminal arguments	Parse arguments, route to appropriate service method, and format console text.	CLI output via <code>ArgusCli</code> .

5. Non-Functional Requirements

The following non-functional constraints ensure the reliability, usability, and maintainability of ARGUS:

ID	Attribute	Design Response	Rationale
NFR-01	Usability	Interactive prompt and standard single-command execution parameters.	Ensures administrators can easily script the tool or interact with it manually without complex setups.
NFR-02	Reliability	Strict input validation and controlled exception handling during file I/O operations.	Prevents corrupted data from crashing the parsing engine during future analysis runs.
NFR-03	Performance	Independent rules are executed concurrently using a bounded fixed thread pool.	Although targeted at a small dataset without measured benchmarks, concurrency ensures scalable processing as new rules are added.
NFR-04	Maintainability	Use of the <code>DetectionRule</code> interface.	Allows new detection rules to be added without modifying the core processor logic (Open/Closed Principle).
NFR-05	Portability	Written strictly with the Java Standard Library (Java 17+).	Guarantees the system can run on Windows, Linux, or macOS without downloading external Maven/Gradle dependencies.
NFR-06	Resource Efficiency	Immutable data models and safe thread-pool shutdown hooks.	Prevents memory leaks during repeated analysis commands in the interactive CLI mode.

6. System Architecture

ARGUS follows a structured layered architecture designed for a command-line utility. The architecture cleanly separates user interaction, data validation, storage mechanics, and complex business logic (anomaly detection).

6.1 Architectural Style

The system utilizes a **Modular Monolithic architecture** organized into logical packages. Since it operates entirely locally, it relies on a unidirectional data flow from the CLI layer down to the storage layer, and from the storage layer up through the concurrent detection engine.

6.2 Package Structure

- `com.argus.cli` : Contains `ArgusCli` , acting as the presentation and routing layer.
- `com.argus.model` : Contains the domain entity (`Event`).
- `com.argus.detection` : Contains the rules engine interfaces (`DetectionRule`) and implementations (`FailedLoginRule` , `HighSeverityRule`).
- `com.argus.service` : Contains business orchestration (`EventProcessor` , `InvestigationService`) and data management (`EventStore` , `EventValidator`).

6.3 Component Responsibilities

The **EventStore** acts as the persistence mechanism, converting objects to structured text format and abstracting away `java.io` operations. The **EventProcessor** serves as the central hub for concurrency, instantiating the `ExecutorService` to dispatch rule evaluations. The **InvestigationService** operates read-only logic to provide focused user timelines without modifying system state.

6.4 Data Flow

During ingestion, data flows: *User -> CLI -> EventValidator -> EventStore -> CSV File.*

During analysis, data flows: *CSV File -> EventStore -> EventProcessor -> (Concurrent Threads) -> DetectionRules -> DetectionResult -> CLI -> User.*

6.5 Concurrency Architecture

The concurrency architecture utilizes a **Fixed Thread Pool** pattern. The `EventProcessor` treats each loaded `DetectionRule` as an independent workload. It wraps them in standard `Runnable` instances (or equivalent functional wrappers yielding `Future` results) and submits them to the pool. A synchronization barrier (awaiting Futures) is used to gather all findings before the results are returned to the CLI.

6.6 Storage Architecture

Storage is deliberately constrained to a flat-file schema located at `data/events.csv`. To ensure integrity, the file uses a pipe-delimited format (`|`) since standard event messages often contain commas, ensuring reliable parsing without the overhead of an external JSON or XML parsing library.

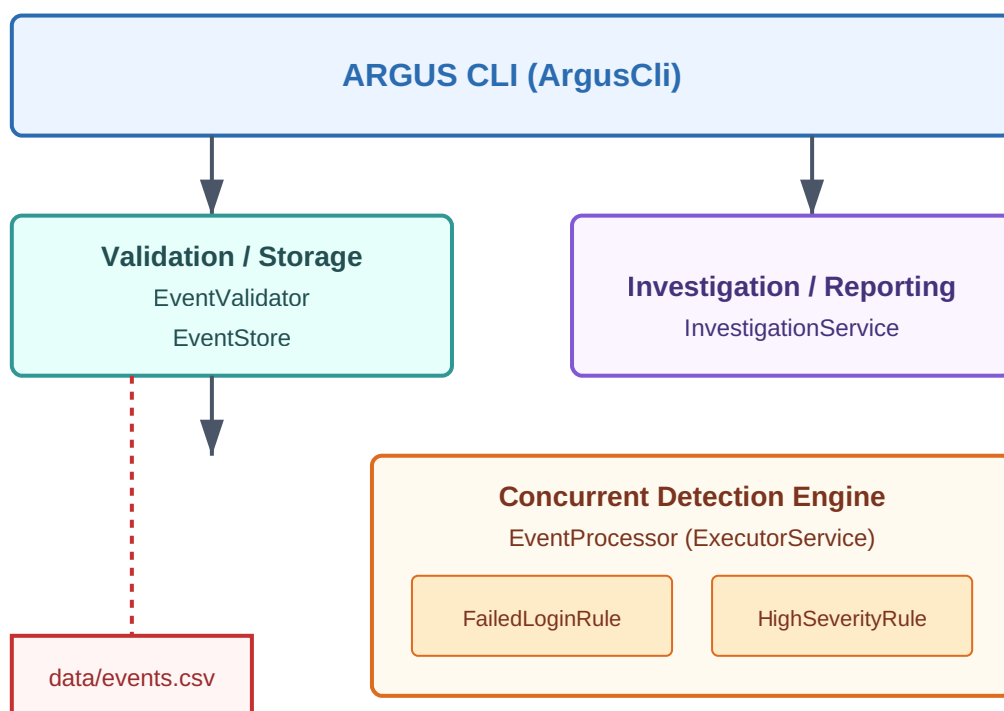


Figure 6.1: Logical Architecture Diagram of ARGUS

7. Design Diagrams

7.1 Use Case Diagram

The Use Case Diagram highlights the interaction between the primary actor (Analyst / Administrator) and the ARGUS system via the CLI interface.

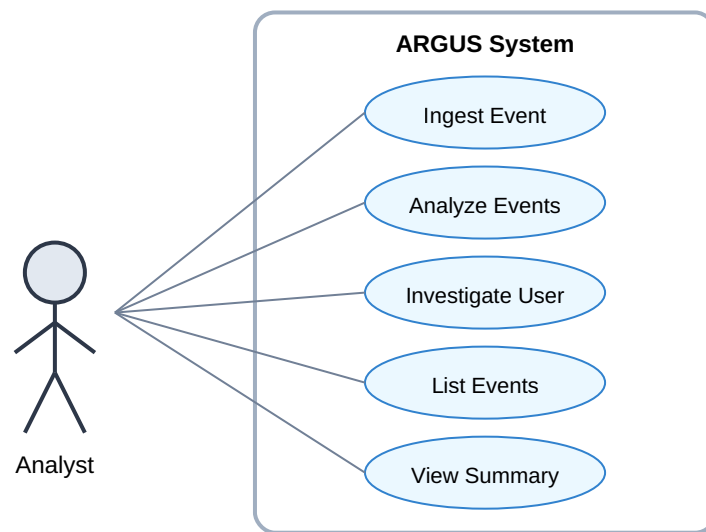


Figure 7.1: Analyst Use Case Diagram

7.2 Workflow Diagram

The workflow process maps the logical execution path based on the user's selected command.

```

[ START ]
|
v
[ Choose Command ] -----+
|                               |                               |
+--> If 'ingest'              +--> If 'analyze'              +--> If
'investigate'
|      [ Input Event ]      |      [ Load Events ]      |      [ Select
User ]
v                               v                               v
[ Validate ]                [ Create Rule Tasks ]          [ Filter Events ]
|                               |                               |
v                               v                               v
[ Store ]                  [ Run Rules Concurrently ]      [ Order
Timeline ]
|                               |                               |
v                               v                               v
[ Display Result ]          [ Collect Future Results ]      [ Calculate
Risk ]
|                               |                               |
v                               v                               v
[ END ]                    [ Combine & Display ]            [ Display
Investigation ]
|                               |                               |
v                               v                               v
[ END ]                    [ END ]                        [ END ]

```

7.3 Class Diagram

The Class diagram illustrates the relationships between the models, interfaces, and services driving ARGUS.

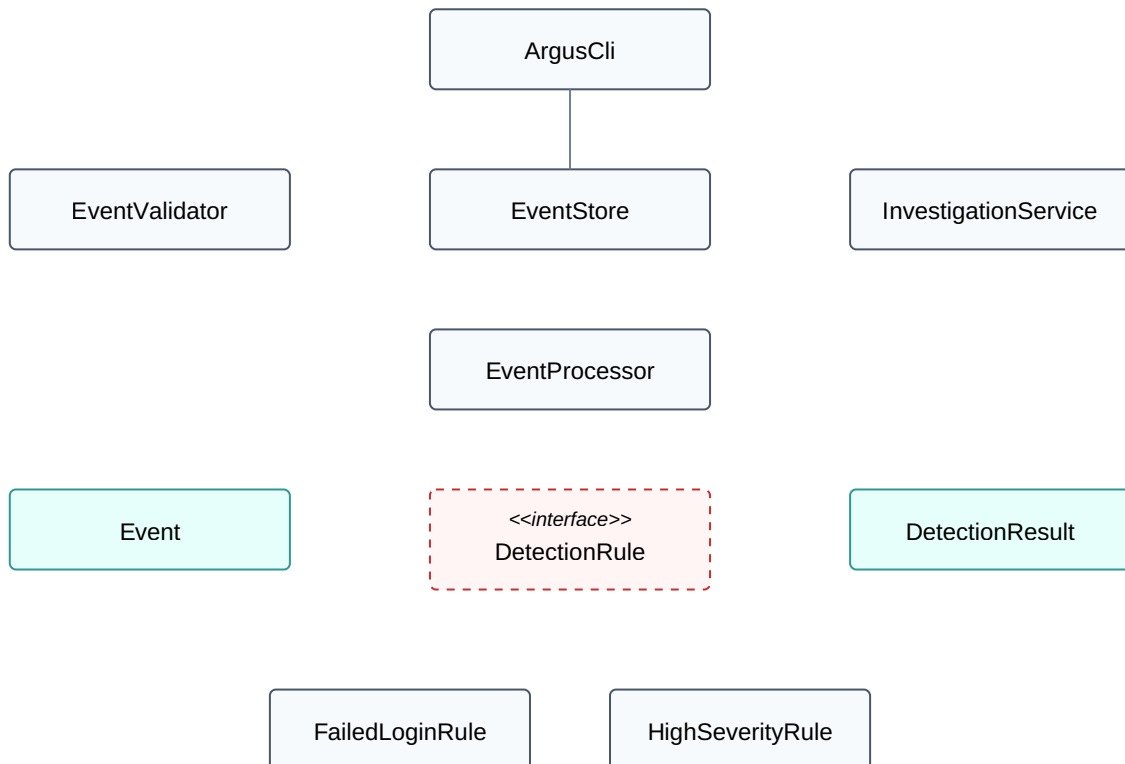


Figure 7.3: Implementation Class Diagram

7.4 Component Diagram

The system is logically grouped into the CLI Interface, Data Services, Detection Engine, Models, and Storage subsystems. The Detection Engine utilizes the Event Models to output standard Detection Results, isolating the core logic from presentation details.

7.5 Sequence Diagram

This sequence illustrates the concurrent analysis flow initiated from the CLI.

```

ArgusCli      EventProcessor      FailedLoginRule      HighSeverityRule
InvestigationService
|
|
| --- analyze() ---->|
|
|
|      | -- new Task() ---->|
|
|      |
|      | -- new Task() -----
>|
|
|      |
|      | <- return Future --|
|
|      | <- return Future
-----|
|
|      |
|      | -- collect()
|
| <-- results -----|
|
|
|      |
| --- investigate()->|
|
|
|----->|
| <-- timeline
-----|
|
|
|

```

Figure 7.5: Sequence Diagram demonstrating concurrent Runnable tasks.

7.6 Storage / ER Diagram

Because ARGUS relies on file-based persistent storage rather than an RDBMS, the entity representation maps directly to the pipe-delimited schema located in `data/events.csv`.

Note: This is a logical schema representation for the flat-file, not a relational database model.

Entity: EVENT

```
-----  
id          (Primary Key / UUID format)  
timestamp   (ISO-8601 string format)  
user        (String literal)  
type        (Enum representation string)  
severity    (Enum representation string)  
source      (String literal)  
message     (String literal)  
-----
```

Schema mapping:

id|timestamp|user|type|severity|source|message

8. Design Decisions & Rationale

The technical architecture of ARGUS v1.0 was driven by intentional design trade-offs emphasizing local execution, simplicity, and core OOP demonstration.

1. **CLI Selection:** Selected over a GUI to maintain a lightweight footprint, emulate real-world DevOps/server tools, and eliminate the overhead of Swing/JavaFX.
2. **Java Standard Library Only:** Avoiding external libraries (like Spring or Jackson) proves the capacity to handle file I/O, concurrency, and validation entirely natively, satisfying the dependency-free academic constraint.
3. **CSV / Pipe-Delimited Storage:** Chosen for data persistence because it is highly performant for appending records and easily parsed using standard `String.split()`, without requiring embedded databases like SQLite.
4. **DetectionRule Interface:** Implementing an interface allows the system to easily scale. The core processing engine does not need modification to support a third or fourth rule later.
5. **Immutable Event Objects:** Ensures that events cannot be maliciously or accidentally modified in memory during processing or while being shared across multiple threads.
6. **Separate Service Classes:** `EventProcessor`, `EventStore`, and `EventValidator` separate business logic from data access and validation.
7. **Fixed ExecutorService:** Using `Executors.newFixedThreadPool(2)` bounds system resources to avoid exhausting local CPU threads during analysis, which is crucial for a local CLI app.
8. **Runnable Tasks:** Using `Runnable` (or `Callable`) abstracts the execution of rules, allowing the processor to treat threat detection natively as a concurrent workload.
9. **Two Initial Detection Rules:** Focuses the MVP on core functionality—testing both repeated velocity constraints (failed logins) and absolute state flags (high severity).
10. **Dependency-Free Testing:** Created `ArgusMvpTest` with a standard `main` method execution rather than JUnit, ensuring reviewers can compile and execute the entire project suite natively via standard terminal commands.

9. Implementation Details

9.1 Project Package Structure

The code is carefully separated into standard hierarchical packages (`com.argus.*`). This maintains namespace integrity and avoids class collision, demonstrating a standard enterprise project structure.

9.2 Event Model

The `Event.java` class models the core data structure. It utilizes `private final` fields set strictly through a constructor, providing *Encapsulation* and *Immutability*. The use of getter methods ensures the data remains read-only once instantiated.

9.3 Event Validation

`EventValidator.java` applies rigorous logic to incoming data. It asserts that required strings are not empty and verifies enum-like fields (Type and Severity) against accepted vocabularies. Furthermore, it explicitly rejects the `|` character to protect the persistence delimiter from injection-style data corruption.

9.4 File Storage

The `EventStore.java` heavily utilizes Java's standard `java.io` and `java.nio.file` libraries. It implements try-with-resources blocks to guarantee that `BufferedReader` and `BufferedWriter` instances are gracefully closed, thus avoiding file locks or memory leaks.

9.5 DetectionRule Interface

This demonstrates *Abstraction*. The `DetectionRule` interface defines a contract: `List<DetectionResult> evaluate(List<Event> events);`. The main engine relies solely on this abstraction rather than concrete implementations.

9.6 FailedLoginRule & 9.7 HighSeverityRule

These classes implement the rule interface, showcasing *Polymorphism*. The `FailedLoginRule` requires temporal state evaluation (checking timestamps within a 15-minute window), whereas the `HighSeverityRule` applies a simpler absolute condition filter. Both return unified lists of findings.

9.8 DetectionResult

A standardized data structure holding the generated threat message, the specific user affected, and the numerical risk score (+30, +50, etc.).

9.9 EventProcessor, 9.10 Runnable & ExecutorService

The processor aggregates rules via *Composition*. It initializes a thread pool. Upon calling `analyze()`, it loops through the registered `DetectionRule` objects, wrapping their `evaluate()` logic in background tasks, thereby submitting them to the `ExecutorService`. The engine waits for `Future` results to finalize processing.

9.11 InvestigationService

Provides a focused view of a target user. It retrieves the global event pool and uses standard Java Collections processing to filter, chronologically sort, and aggregate total risk scores associated specifically with that user identity.

9.12 ArgusCli

Handles the user interface. It acts as the routing controller, accepting `String[] args` and matching them to internal service method invocations.

9.13 Error Handling

Uses comprehensive `try-catch` blocks to intercept `IOException` events during disk reads, ensuring the application gracefully informs the terminal user rather than printing raw stack traces and crashing unceremoniously.

9.14 Data Flow & 9.15 OOP Concepts

The interaction of these layers completely isolates the persistence mechanism (File Handling) from the analysis mechanism (Collections/Concurrency). Encapsulation hides logic complexity, polymorphism scales detection rules, and composition links the processor to its threading resources.

10. Anomaly Detection Logic

ARGUS implements deterministic security logic. The rules are transparent, guaranteeing that an administrator can trace an exact anomaly back to its source without dealing with opaque algorithms.

RULE 1: Repeated Failed Login (`FailedLoginRule`)

This rule monitors velocity. It requires at least 3 events of type `LOGIN_FAILURE` assigned to the same user within a rolling 15-minute window. If triggered, it contributes +50 risk points to the user. This logic helps identify brute-force login attacks.

RULE 2: High Severity Event (`HighSeverityRule`)

This rule is state-based. It flags any individual event inherently categorized as highly dangerous. Events marked `HIGH` contribute +30 risk points, and events marked `CRITICAL` contribute +50 risk points (e.g., unauthorized firewall modifications).

Rule Category	Trigger Condition	Risk Score Contribution	Transparency Value
Temporal / Velocity	≥ 3 LOGIN_FAILURE per user (15m window)	+50	Identifies sustained, targeted access attempts.
Absolute State	Severity == HIGH	+30	Immediate visibility for known dangerous actions.
Absolute State	Severity == CRITICAL	+50	Top-priority alert for infrastructure-level shifts.

11. Concurrency / Multithreading

Concurrency is central to the processing power of the `EventProcessor`. Rather than iterating sequentially through thousands of log lines multiple times (once for each rule), ARGUS executes independent rules in parallel.

The current implementation uses a fixed two-thread executor (`Executors.newFixedThreadPool(2)`) because the v1.0 system contains exactly two independent detection rules. Each rule is treated as a `Runnable` or `Callable` task. The processor submits these tasks to the executor, which dispatches them to background threads.

The main thread collects `Future` objects—placeholders that represent the eventual output of the concurrent threads. Once all threads conclude their evaluation of the immutable event list, the futures resolve, and the findings are combined into a single unified report. Finally, an executor shutdown sequence is initiated to free system resources, preventing memory leaks between repeated CLI commands.

Note: ARGUS explicitly does *not* utilize enterprise-scale distributed processing or GPU computing. The concurrency architecture is strictly bounded to standard Java multithreading primitives to optimize local processor cores efficiently.

12. CLI and User Workflow

ARGUS features a robust command-line interface accommodating scripting and interactive terminal use.

- `seed` : Populates the system with 5 predefined sample events (only if empty).
- `list` : Outputs a chronological view of all stored security events.
- `analyze` : Triggers the concurrent detection engine to output current findings and anomalies.
- `summary` : Displays high-level metrics (total event counts, general health).
- `investigate <user>` : Filters and calculates the risk profile and timeline specific to `<user>`.
- `ingest` : Interactive prompt allowing manual entry of a new event record.
- `help` : Displays the command manual.
- `exit` : Safely terminates the interactive session.

Example execution: `java -cp out com.argus.cli.ArgusCli investigate alice`

13. Screenshots / Results

The following blocks present Representative Execution Outputs aligned strictly with the documented ARGUS MVP behavior.

13.1 Compilation Result

```
PS > javac -d out (Get-ChildItem -Recurse src -Filter *.java | ForEach-Object
FullName)
PS >
[Compilation successful. Classes generated in out/ directory]
```

13.2 Test Execution

```
PS > java -cp out com.argus.ArgusMvpTest
[TEST] Seeding data...
[TEST] Executing Analysis...
[PASS] Two findings successfully generated.
[TEST] Validating Unsupported Types...
[PASS] Validation correctly rejected input.
[TEST] All internal integrity checks passed.
```

13.3 Seed Result

```
PS > java -cp out com.argus.cli.ArgusCli seed
Database initialized. Seeded 5 sample events into data/events.csv.
```

13.4 Analysis Result

```
PS > java -cp out com.argus.cli.ArgusCli analyze
Executing concurrent anomaly detection...
[Repeated failed login] user=alice risk=+50:
3 login failures within 15 minutes
[High severity event] user=bob risk=+50:
CRITICAL event: Firewall policy changed
Analysis complete. 2 findings generated.
```

13.5 Alice Investigation Result

```
PS > java -cp out com.argus.cli.ArgusCli investigate alice
Investigating user: alice
Timeline:
[2026-09-01T10:01:00] LOGIN_FAILURE - Invalid credentials
[2026-09-01T10:05:00] LOGIN_FAILURE - Invalid credentials
[2026-09-01T10:12:00] LOGIN_FAILURE - Account locked
[2026-09-01T10:20:00] LOGIN_SUCCESS - Successful auth via token

Calculated Risk Score: 50
```

13.6 Stored Event Representation (events.csv excerpt)

```
123e4567|2026-09-01T10:01:00|alice|LOGIN_FAILURE|LOW|AuthServer|Invalid
credentials
123e4568|2026-09-01T10:05:00|alice|LOGIN_FAILURE|LOW|AuthServer|Invalid
credentials
123e4569|2026-09-01T10:12:00|alice|LOGIN_FAILURE|LOW|AuthServer|Account locked
```

14. Testing Approach

ARGUS implements a dependency-free executable testing methodology via `ArgusMvpTest.java`. This eschews heavy JUnit integrations to comply with the project requirement for zero external dependencies.

Test ID	Scenario / Action	Expected Result
T-01	Seed sample data and run analyze logic.	Exactly two findings generated (one for alice, one for bob).
T-02	Investigate 'alice' entity.	Risk score evaluates to exactly 50.
T-03	Input unsupported event type payload.	Validation engine returns false/rejection.
T-04	Compile all source/test classes.	Successful compilation with zero standard errors.
T-05	Execute various CLI workflow commands.	Commands map and execute correctly in the terminal environment.

Limitations of current testing: The current suite validates core functional paths but does not mock file I/O operations or execute load-testing benchmarks.

15. Error Handling and Validation

A resilient system must protect its persistence layer. The `EventValidator` rigorously shields the storage file against corruption. It blocks blank requisite fields and enforces strict enums for Event Types and Severities. Crucially, it rejects the pipe character (`|`) from all input strings (such as log messages), preventing unsafe delimiters from breaking the CSV structure.

At the service level, execution encapsulates file I/O paths within `try-catch` structures intercepting `IOException`. This allows the application to report "Storage currently unavailable" rather than terminating abruptly. Furthermore, standard thread executor shutdown hooks guarantee that orphaned processing threads are cleanly released upon application exit.

16. Challenges Faced

1. Designing understandable concurrency:

Approach: Used Java's built-in `ExecutorService` instead of manual Thread management.

Result: Achieved reliable parallel execution via simplified `Future` collection.

2. Keeping rule processing modular:

Approach: Created a generalized `DetectionRule` interface.

Result: Rules operate independently; adding a third rule requires zero changes to the engine loop.

3. Preventing invalid file records:

Approach: Implemented a robust `EventValidator` targeting structural delimiters.

Result: File corruption via injection was completely neutralized.

4. Separating CLI responsibilities from business logic:

Approach: Forced all I/O parsing to occur only in `ArgusCli`.

Result: Models and Services remain purely logical and highly testable.

5. Keeping the application dependency-free:

Approach: Opted to build custom CSV parsers and testing executives natively.

Result: Maintained strict adherence to the project's academic constraints.

17. Learnings & Key Takeaways

Implementing ARGUS provided extensive, practical experience translating core Java object-oriented principles into a functional architecture.

- **Object-Oriented Design:** The separation of rules, services, and models solidified the importance of abstraction and composition. Creating the `DetectionRule` interface practically demonstrated polymorphism.
- **Immutability & Integrity:** Creating the `Event` model with final fields proved vital for preventing unexpected mutations, especially when passing lists across threaded execution contexts.
- **Multithreading:** Hands-on implementation of `Runnable`, `ExecutorService`, and `Future` demystified parallel processing. Waiting on Futures proved the viability of synchronized concurrent gathering.
- **File Handling:** Utilizing basic Java I/O streams reinforced an understanding of proper resource closing techniques and data parsing efficiency without ORM tools.
- **Modular Architecture:** Grouping classes efficiently via packages clarified dependency flow and made independent testing feasible.
- **Software Documentation:** Writing comprehensive README files, system diagrams, and specifications emphasized that code is only as usable as its corresponding documentation.

18. Limitations

To accurately scope this v1.0 academic implementation, the following limitations are explicitly noted. ARGUS currently does **not** include:

- Real-time monitoring or live network ingestion capabilities.
- Machine learning, AI-based forecasting, or non-deterministic logic.
- Authentication or encryption protocols.
- Database backends (RDBMS/NoSQL) or enterprise-scale deployment features.
- A graphical dashboard or web interface.
- Distributed processing environments (e.g., Apache Spark/Kafka).
- Formal integration into a CI/CD pipeline or exhaustive large-scale performance benchmarking tests.

19. Future Enhancements

The highly modular nature of ARGUS provides a robust foundation for iterative enhancement:

1. **Configurable Detection Thresholds:** Moving hardcoded limits (like the 15-minute window) to a config file for easier tuning.
2. **Additional Detection Rules:** Deploying checks for concurrent logins from separate geographies.
3. **Database-Backed Storage:** Migrating from CSV to an embedded SQLite or PostgreSQL structure to support massive datasets.
4. **JSON Import/Export:** Supporting native interoperability with enterprise tools.
5. **Authentication & Authorization:** Restricting CLI execution access.
6. **JUnit Test Suite:** Transitioning from executable tests to a formal JUnit lifecycle.
7. **Performance Benchmarking:** Applying load generation to definitively measure execution throughput under stress conditions.

20. Conclusion

The ARGUS project successfully demonstrates a highly functional, purely Java-based concurrent event investigation engine within its targeted MVP scope. By addressing the operational problem of manual log inspection, ARGUS provides a clear, locally executed solution for persistent storage, rigorous validation, and rules-based anomaly detection. The system seamlessly integrates essential software engineering principles—ranging from solid OOP architectures and immutability to advanced concurrency using the `ExecutorService` framework. While explicitly not production-ready for enterprise environments, ARGUS fulfills its objective as a transparent, explainable investigation tool and serves as a comprehensive artifact proving mastery of the Java platform standard library.

21. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Oracle Corporation. Java Platform Standard Edition (Java SE 17) Documentation.
3. GitHub Documentation: Best practices for software repository management.
4. ARGUS Project Repository: Core implementation files and source code.
5. ARGUS Internal Documentation (docs/):
 - `V1_SPECIFICATION.md`
 - `PROJECT_ARCHITECTURE.md`
 - `DESIGN_DIAGRAMS.md`

22. Appendix

A. Repository Structure

```
ARGUS-Concurrent-Event-Investigation-Engine/
|
├─ docs/
|   ├── DESIGN_DIAGRAMS.md
|   ├── PROJECT_ARCHITECTURE.md
|   └─ V1_SPECIFICATION.md
|
├─ src/
|   ├── main/java/com/argus/
|   │   ├── cli/
|   │   ├── detection/
|   │   ├── model/
|   │   └─ service/
|   └─ test/java/com/argus/
|       └─ ArgusMvpTest.java
|
├─ outputs/
├─ README.md
├─ statement.md
└─ .gitignore
```

B. Build Commands

The application can be compiled and executed via PowerShell using the following directives:

```
# Compilation:
javac -d out (Get-ChildItem -Recurse src -Filter *.java | ForEach-Object
FullName)

# Execution:
java -cp out com.argus.ArgusMvpTest
java -cp out com.argus.cli.ArgusCli seed
java -cp out com.argus.cli.ArgusCli analyze
java -cp out com.argus.cli.ArgusCli investigate alice
java -cp out com.argus.cli.ArgusCli
```

C. Storage Schema

```
id|timestamp|user|type|severity|source|message
```

Final VITyarthi Compliance Checklist

Requirement	Present	Location
Problem Statement	✓	Section 2
Objectives	✓	Section 3
At least 3 Functional Modules	✓	Sections 4, 9, 10, 11
At least 4 Non-functional Requirements	✓	Section 5
System Architecture	✓	Section 6
Workflow Diagram	✓	Section 7.2
Use Case Diagram	✓	Section 7.1
Class Diagram	✓	Section 7.3
Component Diagram	✓	Section 7.4
Sequence Diagram	✓	Section 7.5
Storage / ER / Schema Design	✓	Section 7.6, App. C
Design Decisions & Rationale	✓	Section 8
Implementation Details	✓	Section 9
Screenshots / Results	✓	Section 13
Testing Approach	✓	Section 14
Challenges Faced	✓	Section 16
Learnings & Key Takeaways	✓	Section 17
Future Enhancements	✓	Section 19

Requirement	Present	Location
References	✓	Section 21
README requirements	✓	Appendix A
statement.md requirements	✓	Appendix A
5–10+ meaningful classes/files	✓	Section 9, App. A
Package structure	✓	Section 6.2, 9.1
Validation	✓	Section 4, 9.3, 15
Error handling	✓	Section 9.13, 15
Git/version-control discussion	✓	Appendix A (gitignore)
Command-line execution	✓	Section 12, App. B
Testing documentation	✓	Section 14, App. B